

INSTITUTO FEDERAL DO TRIÂNGULO MINEIRO

Campus Uberlândia Centro

Estruturas de Dados — 2026/1

APOSTILA — UNIDADE IV

Aplicações em Sistemas Web com Java

Curso Superior de Tecnologia em Sistemas para Internet

1. Objetivos de Aprendizagem

Ao concluir esta unidade, o estudante será capaz de:

- Utilizar o JMH (Java Microbenchmark Harness) para medir o desempenho de estruturas de dados e identificar gargalos em aplicações Java.
- Compreender como bancos de dados relacionais utilizam árvores B e B+ para indexação e relacionar esse conhecimento ao comportamento do JPA/Hibernate.
- Implementar e configurar caching em aplicações Spring Boot utilizando Spring Cache, Redis e estratégias como LRU Cache com LinkedHashMap.
- Compreender o modelo pub/sub e processar mensagens de forma assíncrona com RabbitMQ e Spring AMQP.
- Aplicar padrões de uso de estruturas de dados (LRU Cache, Rate Limiting, autocompletar com Trie) em contextos reais de API REST.
- Escolher a estrutura de dados mais adequada para cada cenário de API REST de alto desempenho.
- Desenvolver um projeto integrador completo com análise de performance, benchmarking e documentação técnica.

2. Contexto Profissional

Imagine que você foi contratado como desenvolvedor backend sênior em uma fintech que processa 500.000 transações por dia. A plataforma é construída sobre Spring Boot e apresenta os seguintes problemas reportados pelo time de SRE (Site Reliability Engineering):

- Latência média de 800 ms em consultas de saldo — aceitável é até 150 ms.
- Pico de 95% de uso de CPU durante reconciliação noturna de dados.
- Cache de sessão expirado gerando 40% de cache miss, sobrecarregando o banco de dados.
- Filas de notificação de transações causando timeout em endpoints síncronos.

Cada tópico desta unidade resolve diretamente um desses problemas:

Problema	Solução abordada nesta unidade
Latência alta nas consultas	JMH para identificar gargalos + escolha correta de estrutura
CPU alta na reconciliação	Indexação com árvores B/B+ (JPA/Hibernate) + queries otimizadas
Cache miss elevado	Spring Cache + Redis + LRU Cache com LinkedHashMap
Timeouts em notificações	RabbitMQ + Spring AMQP para processamento assíncrono

Esta unidade é, portanto, uma síntese aplicada: cada estrutura de dados estudada nas unidades anteriores (listas, árvores, heaps, hash tables) reaparece aqui em um contexto profissional real, com ferramentas e frameworks utilizados pelo mercado.

3. Fundamentos Teóricos

3.1 Profiling e Benchmarking em Java

Benchmarking é o processo de medir o desempenho de um fragmento de código de forma controlada e repetível. Profiling é a análise dinâmica de uma aplicação em execução para identificar onde o tempo de CPU e memória são consumidos.

3.1.1 Por que não usar `System.nanoTime()`?

Uma medição ingênua com `System.nanoTime()` sofre de várias distorções:

- **JIT Compilation:** o compilador JIT (Just-In-Time) da JVM otimiza o código em tempo de execução. As primeiras execuções são interpretadas; as subsequentes são compiladas. Medir as primeiras é enganoso.
- **Aquecimento da JVM (warm-up):** a JVM carrega classes, aloca estruturas internas e realiza profiling próprio nas primeiras iterações.
- **Dead Code Elimination:** o compilador pode eliminar código cujo resultado nunca é utilizado, tornando o benchmark inválido.
- **Garbage Collector:** pausas de GC podem inflar as medições de forma imprevisível.

```
// ❌ Benchmark INGÊNUO — não confiável
long inicio = System.nanoTime();
minhaEstrutura.buscar("chave");
long fim = System.nanoTime();
System.out.println("Tempo: " + (fim - inicio) + " ns");
// Problemas: sem warm-up, sem repetições, sem controle de GC
```

3.1.2 JMH — Java Microbenchmark Harness

O JMH é a ferramenta oficial para microbenchmarking em Java, desenvolvida pela OpenJDK. Ele resolve todos os problemas acima de forma transparente:

- **Fork:**
- **Warm-up:** cada benchmark é executado em um processo JVM separado para isolamento.
- **Blackhole:** JMH executa iterações de aquecimento antes de medir.
- impede que o compilador elimine código cujo resultado "nunca é usado".
- **Modos de medição:**

Configuração Maven (pom.xml):

```
<dependency>
  <groupId>org.openjdk.jmh</groupId>
  <artifactId>jmh-core</artifactId>
  <version>1.37</version>
</dependency>
<dependency>
  <groupId>org.openjdk.jmh</groupId>
  <artifactId>jmh-generator-annprocess</artifactId>
  <version>1.37</version>
  <scope>provided</scope>
</dependency>
```

Estrutura básica de um benchmark JMH:

```

import org.openjdk.jmh.annotations.*;
import org.openjdk.jmh.infra.Blackhole;
import java.util.concurrent.TimeUnit;

@BenchmarkMode(Mode.AverageTime)          // mede tempo médio por operação
@OutputTimeUnit(TimeUnit.NANOSECONDS)     // unidade de saída
@State(Scope.Benchmark)                   // estado compartilhado entre
threads
@Warmup(iterations = 5, time = 1)          // 5 iterações de aquecimento de
1s cada
@Measurement(iterations = 10, time = 1)    // 10 iterações de medição
@Fork(2)                                   // 2 processos JVM separados
public class EstruturasBenchmark {

    // Estado preparado antes das medições
    private HashMap<String, Integer> hashMap;
    private TreeMap<String, Integer> treeMap;

    @Setup(Level.Trial)
    public void setup() {
        hashMap = new HashMap<>();
        treeMap = new TreeMap<>();
        for (int i = 0; i < 10_000; i++) {
            String chave = "chave-" + i;
            hashMap.put(chave, i);
            treeMap.put(chave, i);
        }
    }

    @Benchmark
    public void buscaHashMap(Blackhole bh) {
        // Blackhole "consume" o resultado — evita dead code elimination
        bh.consume(hashMap.get("chave-5000"));
    }

    @Benchmark
    public void buscaTreeMap(Blackhole bh) {
        bh.consume(treeMap.get("chave-5000"));
    }
}

```

Modos de medição disponíveis no JMH:

Mode	Descrição
AverageTime	Tempo médio por operação — mais comum para latência
Throughput	Operações por segundo — para medir vazão
SampleTime	Distribuição de tempos (percentis p99, p999)
SingleShotTime	Tempo de uma única execução — para cold start

3.1.3 VisualVM e GC Logs

Para análise de aplicações em execução (profiling), o VisualVM é a ferramenta padrão do ecossistema Java. Permite:

- Monitorar uso de CPU e heap em tempo real.
- Realizar CPU profiling para identificar os métodos mais custosos (hot methods).

- Analisar o heap dump para detectar vazamentos de memória.
- Monitorar threads e detectar deadlocks.

Para habilitar logs de GC em aplicações Spring Boot, adicione na linha de comando JVM:

```
# JDK 17+ (Unified GC Logging)
java -Xlog:gc*:file=gc.log:time,uptime:filecount=5,filesize=20m -jar
app.jar

# Flags úteis para análise de pressão de memória:
# -Xms512m -Xmx2g          -> define heap mínimo e máximo
# -XX:+UseG1GC             -> usa o G1 Garbage Collector (padrão JDK 17+)
# -XX:MaxGCPauseMillis=200 -> meta de pausa máxima do GC
```

Indicadores críticos nos logs de GC:

- Frequência de GC: se o GC roda mais de 1x/segundo, há pressão de memória.
- Duração das pausas: pausas acima de 200ms afetam latência de APIs REST.
- Crescimento do Old Generation: indica possível vazamento de memória.

3.2 Estruturas de Dados em Bancos de Dados — Árvores B e B+

Quando você faz uma consulta SQL como `SELECT * FROM transacoes WHERE conta_id = 12345`, o banco de dados não varre linha por linha ($O(n)$). Ele utiliza um índice, que internamente é implementado como uma árvore B+ — uma variação balanceada da árvore B otimizada para armazenamento em disco.

3.2.1 Árvore B — Conceitos Fundamentais

Uma árvore B de ordem m é uma árvore de busca balanceada onde:

- Cada nó tem no máximo m filhos e $m-1$ chaves.
- Cada nó interno (exceto raiz) tem pelo menos $\lceil m/2 \rceil$ filhos.
- Todas as folhas estão no mesmo nível — garantia de balanceamento.
- As chaves em cada nó são ordenadas.

A motivação para árvores B é o custo de I/O em disco. Um nó de árvore B pode armazenar centenas de chaves (equivalente a uma página de disco), minimizando o número de leituras de disco por consulta.

3.2.2 Árvore B+ — A Estrutura Real dos Índices

A árvore B+ é a variante utilizada pela maioria dos SGBDs (PostgreSQL, MySQL InnoDB, Oracle):

- Todos os registros (pares chave → endereço) ficam nas folhas.
- Os nós internos armazenam apenas chaves de roteamento — sem dados.
- As folhas são ligadas em lista duplamente encadeada, permitindo range scans eficientes (BETWEEN, ORDER BY).
- Complexidade de busca: $\Theta(\log_m n)$, onde m é a ordem da árvore (tipicamente 100–1000).

Impacto prático: uma árvore B+ de ordem 500 armazenando 1 bilhão de registros tem altura máxima de $\log_{500}(10^9) \approx 3$. Isso significa apenas 3 leituras de disco para qualquer busca.

3.2.3 Relação com JPA/Hibernate

Cada anotação `@Index` no JPA instrui o banco a criar uma árvore B+ para aquela coluna:

```

@Entity
@Table(name = "transacoes", indexes = {
    // Cria árvore B+ na coluna conta_id - busca O(log n)
    @Index(name = "idx_conta_id", columnList = "conta_id"),
    // Índice composto - otimiza queries com WHERE conta_id AND data
    @Index(name = "idx_conta_data", columnList = "conta_id,
data_transacao")
})
public class Transacao {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "conta_id", nullable = false)
    private Long contaId;

    @Column(name = "data_transacao")
    private LocalDateTime dataTransacao;

    @Column(name = "valor", precision = 15, scale = 2)
    private BigDecimal valor;
}

// Repositório com queries otimizadas pelos índices:
public interface TransacaoRepository extends JpaRepository<Transacao,
Long> {

    // Usa o índice idx_conta_id - percorre a árvore B+
    List<Transacao> findByContaId(Long contaId);

    // Usa o índice idx_conta_data - range scan na árvore B+
    @Query("SELECT t FROM Transacao t WHERE t.contaId = :id " +
        "AND t.dataTransacao BETWEEN :inicio AND :fim")
    List<Transacao> buscarPorContaEPeriodo(
        @Param("id") Long contaId,
        @Param("inicio") LocalDateTime inicio,
        @Param("fim") LocalDateTime fim
    );
}

```

⚠ Armadilha: N+1 Query Problem

O problema N+1 é o bug de performance mais comum com Hibernate. Ao buscar N entidades pai, o Hibernate faz N queries adicionais para carregar entidades filha com LAZY loading.

Solução: usar `@EntityGraph` ou `JOIN FETCH` nas queries críticas.

3.3 Caching em Aplicações Java

Caching é a técnica de armazenar resultados de operações custosas em memória rápida para reutilização posterior. Em sistemas web, caching pode reduzir a latência de consultas de 500 ms para < 1 ms.

3.3.1 Spring Cache — Abstração de Alto Nível

Spring Cache é uma abstração que separa a lógica de negócio do mecanismo de cache. Você anota métodos e o Spring gerencia o cache automaticamente:

```
// 1. Habilitar cache na aplicação
@SpringBootApplication
@EnableCaching
public class FinanceiroApp { ... }

// 2. Configurar o provider (Redis neste exemplo)
@Configuration
public class CacheConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory factory)
    {
        RedisCacheConfiguration config =
RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))           // TTL de 10 minutos
            .serializeValuesWith(
                RedisSerializationContext.SerializationPair
                    .fromSerializer(new
GenericJackson2JsonRedisSerializer())
            );
        return RedisCacheManager.builder(factory)
            .cacheDefaults(config)
            .build();
    }
}

// 3. Usar as anotações nos services
@Service
public class SaldoService {

    @Autowired
    private SaldoRepository repository;

    // @Cacheable: retorna do cache se existir; executa e armazena se não
    existir
    // key = SpEL expression - usa o parâmetro contaId como chave
    @Cacheable(value = "saldos", key = "#contaId")
    public BigDecimal buscarSaldo(Long contaId) {
        // Este código só executa se não há cache para #contaId
        return repository.findById(contaId)
            .map(Conta::getSaldo)
            .orElseThrow(() -> new ContaNaoEncontradaException(contaId));
    }

    // @CachePut: SEMPRE executa e atualiza o cache
    @CachePut(value = "saldos", key = "#conta.id")
    public Conta atualizarSaldo(Conta conta) {
        return repository.save(conta);
    }

    // @CacheEvict: remove do cache após execução
    @CacheEvict(value = "saldos", key = "#contaId")
    public void invalidarCacheSaldo(Long contaId) {
        // Não precisa de corpo - o Spring remove do cache
    }
}
```



```

// Remover TODO o cache de saldos de uma vez
@CacheEvict(value = "saldos", allEntries = true)
public void limparTodosOsSaldos() { }
}

```

3.3.2 Redis — Cache Distribuído

Redis (Remote Dictionary Server) é um banco de dados em memória utilizado como cache distribuído. Principais características:

- Suporte a estruturas ricas: strings, hashes, listas, sets, sorted sets, bitmaps, HyperLogLog.
- Persistência opcional: RDB snapshots e AOF (Append-Only File).
- Publicação/Assinatura (pub/sub) nativo.
- TTL (Time-To-Live) por chave.
- Latência < 1 ms para operações em memória.

Configuração Spring Boot + Redis:

```

# application.properties
spring.data.redis.host=localhost
spring.data.redis.port=6379
spring.data.redis.password=${REDIS_PASSWORD}
spring.data.redis.timeout=2000ms

# Uso direto do RedisTemplate para operações avançadas
@Component
public class SessionStore {

    @Autowired
    private RedisTemplate<String, Object> redisTemplate;

    // Armazenar sessão com TTL de 30 minutos
    public void salvarSessao(String sessionId, Usuario usuario) {
        String chave = "sessao:" + sessionId;
        redisTemplate.opsForValue().set(chave, usuario,
Duration.ofMinutes(30));
    }

    // Recuperar sessão
    public Optional<Usuario> buscarSessao(String sessionId) {
        Object valor = redisTemplate.opsForValue().get("sessao:" +
sessionId);
        return Optional.ofNullable((Usuario) valor);
    }

    // Incrementar contador (atômico) - para Rate Limiting
    public Long incrementarRequisicoes(String clienteId) {
        String chave = "rate:" + clienteId;
        Long count = redisTemplate.opsForValue().increment(chave);
        if (count == 1) {
            // Primeira requisição - define TTL de 1 minuto
            redisTemplate.expire(chave, Duration.ofMinutes(1));
        }
        return count;
    }
}

```

3.3.3 LRU Cache com LinkedHashMap

Para caching local (dentro da JVM, sem Redis), a implementação mais elegante usa LinkedHashMap com política de acesso LRU (Least Recently Used). LinkedHashMap mantém a ordem de inserção ou de acesso, permitindo identificar facilmente o elemento menos recentemente acessado.

```
/**
 * Cache LRU thread-safe implementado com LinkedHashMap.
 * Capacidade máxima configurável; quando cheia, remove o elemento
 * acessado há mais tempo (Least Recently Used).
 *
 * @param <K> tipo da chave
 * @param <V> tipo do valor
 */
public class LRUCache<K, V> extends LinkedHashMap<K, V> {

    private final int capacidadeMaxima;

    /**
     * @param capacidade número máximo de entradas
     */
    public LRUCache(int capacidade) {
        // accessOrder = true: reordena ao acessar (get) – mantém LRU na
frente
        super(capacidade, 0.75f, /* accessOrder */ true);
        this.capacidadeMaxima = capacidade;
    }

    /**
     * Chamado automaticamente pelo LinkedHashMap após cada put.
     * Remove o elemento mais antigo quando a capacidade é excedida.
     */
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacidadeMaxima;
    }

    // Wrapper thread-safe
    public static <K, V> Map<K, V> create(int capacidade) {
        return Collections.synchronizedMap(new LRUCache<>(capacidade));
    }
}

// Uso em um serviço Spring:
@Service
public class ProdutoService {

    // Cache local com capacidade de 1000 produtos
    private final Map<Long, Produto> cacheLocal = LRUCache.create(1000);

    public Produto buscarProduto(Long id) {
        // computeIfAbsent: retorna do cache ou busca e armazena
        return cacheLocal.computeIfAbsent(id, this::buscarNoBanco);
    }

    private Produto buscarNoBanco(Long id) {
        // Simulação de query ao banco (lenta)
        return produtoRepository.findById(id).orElseThrow();
    }
}
```

💡 LRU Cache vs. Redis: quando usar cada um?

LRU Cache (LinkedHashMap): cache local, single-instance, dados não críticos, latência ultrabaixa.

Redis: cache distribuído, múltiplas instâncias do serviço, dados compartilhados, TTL controlado.

Em microsserviços com Kubernetes, sempre prefira Redis — cada pod tem sua própria JVM.

3.4 Filas de Mensagens com RabbitMQ e Spring AMQP

3.4.1 O Modelo Pub/Sub e Processamento Assíncrono

Em sistemas de alta carga, operações como envio de e-mail, geração de PDF e notificações não devem bloquear a resposta HTTP. O padrão pub/sub (publish/subscribe) resolve isso:

- Publisher (produtor): publica uma mensagem em uma fila e retorna imediatamente.
- Consumer (consumidor): processa a mensagem de forma assíncrona, sem bloquear o producer.
- Desacoplamento: producer e consumer não precisam estar ativos simultaneamente.

Conceitos fundamentais do RabbitMQ (AMQP):

Conceito	Descrição
Exchange	Recebe mensagens dos producers e as roteia para filas com base em regras (routing key).
Queue	Buffer que armazena mensagens até que um consumer as processe.
Binding	Ligação entre Exchange e Queue, define quais mensagens chegam a qual fila.
Routing Key	Chave de roteamento que determina para qual fila a mensagem é enviada.
Acknowledgment	Confirmação do consumer: a mensagem só é removida da fila após o ACK.

3.4.2 Configuração Spring AMQP

```
// Dependência Maven:
// <artifactId>spring-boot-starter-amqp</artifactId>

// 1. Configuração da fila e exchange
@Configuration
public class RabbitMQConfig {

    public static final String FILA_NOTIFICACOES =
"notificacoes.transacao";
    public static final String EXCHANGE_TRANSACOES =
"transacoes.exchange";
    public static final String ROUTING_KEY = "transacao.nova";

    @Bean
    public Queue filaNotificacoes() {
        // durable=true: fila sobrevive a reinicializações do RabbitMQ
        return QueueBuilder.durable(FILA_NOTIFICACOES).build();
    }
}
```

```

@Bean
public TopicExchange exchangeTransacoes() {
    return new TopicExchange(EXCHANGE_TRANSACOES);
}

@Bean
public Binding binding(Queue filaNotificacoes, TopicExchange
exchangeTransacoes) {
    return BindingBuilder
        .bind(filaNotificacoes)
        .to(exchangeTransacoes)
        .with(ROUTING_KEY);
}

@Bean
public Jackson2JsonMessageConverter messageConverter() {
    return new Jackson2JsonMessageConverter();
}
}

// 2. Producer – envia mensagem e retorna imediatamente
@Service
public class TransacaoService {

    @Autowired
    private RabbitTemplate rabbitTemplate;

    public Transacao processarTransacao(TransacaoDTO dto) {
        Transacao transacao = salvarTransacao(dto);    // operação
síncrona

        // Publicar evento de forma assíncrona – não bloqueia a resposta
HTTP
        rabbitTemplate.convertAndSend(
            RabbitMQConfig.EXCHANGE_TRANSACOES,
            RabbitMQConfig.ROUTING_KEY,
            new EventoTransacao(transacao.getId(), transacao.getValor())
        );

        return transacao;    // retorna sem esperar o processamento da
notificação
    }
}

// 3. Consumer – processa a mensagem em background
@Component
public class NotificacaoConsumer {

    @Autowired
    private EmailService emailService;

    @RabbitListener(queues = RabbitMQConfig.FILA_NOTIFICACOES)
    public void processarNotificacao(EventoTransacao evento) {
        // Executa em thread separada, de forma assíncrona
        emailService.enviarComprovante(evento.getTransacaoId());
        // Se este método lançar exceção, a mensagem é rejeitada (NACK)
        // e pode ser reenqueueada ou enviada para DLQ (Dead Letter
Queue)
    }
}

```

```
}  
}
```

3.5 Padrões de Uso de Estruturas de Dados

3.5.1 Rate Limiting com estruturas de dados

Rate limiting controla quantas requisições um cliente pode fazer em um intervalo de tempo. Existem dois algoritmos clássicos implementáveis com estruturas básicas:

- **Fixed Window Counter:**

Usa um `HashMap<String, AtomicInteger>` mapeando `clienteId` → contador. Simples, mas sujeito a burst no limite da janela.

- **Sliding Window Log:**

Usa um `Deque<Long>` por cliente armazenando timestamps das requisições. Mais preciso, mas maior uso de memória.

```
// Sliding Window Rate Limiter com Deque  
@Component  
public class RateLimiter {  
  
    // Mapa de clienteId -> fila de timestamps das requisições  
    private final Map<String, Deque<Long>> janelas =  
        new ConcurrentHashMap<>();  
  
    private final int MAX_REQUISICOES = 100;           // máximo por  
janela  
    private final long JANELA_MS = 60_000L;           // janela de 1  
minuto  
  
    /**  
     * Verifica se o cliente pode fazer a requisição.  
     * @return true se dentro do limite, false se excedeu  
     */  
    public boolean permitir(String clienteId) {  
        long agora = System.currentTimeMillis();  
  
        // computeIfAbsent cria a fila se não existir (thread-safe com  
ConcurrentHashMap)  
        Deque<Long> timestamps = janelas.computeIfAbsent(  
            clienteId, k -> new ArrayDeque<>()  
        );  
  
        synchronized (timestamps) {  
            // Remove timestamps fora da janela deslizante  
            while (!timestamps.isEmpty() &&  
                agora - timestamps.peekFirst() > JANELA_MS) {  
                timestamps.pollFirst();  
            }  
  
            if (timestamps.size() >= MAX_REQUISICOES) {  
                return false;    // limite excedido  
            }  
  
            timestamps.addLast(agora);    // registra esta requisição  
            return true;  
        }  
    }  
}
```

```

}

// Uso como filtro Spring:
@Component
public class RateLimitFilter extends OncePerRequestFilter {

    @Autowired
    private RateLimiter rateLimiter;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException {

        String clienteId = request.getHeader("X-Client-ID");

        if (clienteId == null || !rateLimiter.permitir(clienteId)) {
            response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value()); //
429
            response.getWriter().write("Rate limit excedido.");
            return;
        }
        chain.doFilter(request, response);
    }
}

```

3.5.2 Autocompletar com Trie

A Trie (árvore de prefixos) é a estrutura ideal para funcionalidades de autocompletar. Cada nó representa um caractere; o caminho da raiz até um nó terminal representa uma palavra completa.

```

public class Trie {

    private static class No {
        Map<Character, No> filhos = new HashMap<>();
        boolean fimDePalavra = false;
        String valorCompleto;    // armazena o termo completo na folha
    }

    private final No raiz = new No();

    /** Insere uma palavra na Trie. Complexidade:  $\Theta(|palavra|)$  */
    public void inserir(String palavra) {
        No atual = raiz;
        for (char c : palavra.toCharArray()) {
            atual = atual.filhos.computeIfAbsent(c, k -> new No());
        }
        atual.fimDePalavra = true;
        atual.valorCompleto = palavra;
    }

    /** Retorna até maxResultados palavras que começam com prefixo. */
    public List<String> autocompletar(String prefixo, int maxResultados)
    {
        No no = buscarNo(prefixo);
        if (no == null) return Collections.emptyList();

        List<String> resultados = new ArrayList<>();
        coletarPalavras(no, resultados, maxResultados);
    }
}

```

```

        return resultados;
    }

    private No buscarNo(String prefixo) {
        No atual = raiz;
        for (char c : prefixo.toCharArray()) {
            atual = atual.filhos.get(c);
            if (atual == null) return null;
        }
        return atual;
    }

    private void coletarPalavras(No no, List<String> resultados, int max)
    {
        if (resultados.size() >= max) return;
        if (no.fimDePalavra) resultados.add(no.valorCompleto);
        for (No filho : no.filhos.values()) {
            coletarPalavras(filho, resultados, max);
        }
    }
}

// Endpoint REST de autocompletar:
@RestController
@RequestMapping("/api/produtos")
public class ProdutoController {

    private final Trie indiceProdutos = new Trie();

    @PostConstruct
    public void carregarProdutos() {
        produtoRepository.findAll().forEach(p ->
        indiceProdutos.inserir(p.getNome()));
    }

    @GetMapping("/autocompletar")
    public List<String> autocompletar(@RequestParam String q) {
        return indiceProdutos.autocompletar(q.toLowerCase(), 10);
    }
}

```

3.6 Escolha de Estruturas de Dados para APIs REST

A escolha da estrutura de dados correta pode ter impacto de ordem de grandeza no desempenho de uma API. A tabela abaixo resume as decisões mais comuns:

Cenário	Estrutura	Complexidade	Justificativa
Busca por ID/chave única	HashMap / Redis	$\Theta(1)$ amort.	Hash table — acesso direto
Busca por intervalo ou ordenação	TreeMap	$\Theta(\log n)$	Red-Black Tree — ordenado
Fila de processamento FIFO	ArrayDeque	$\Theta(1)$ amort.	Melhor que LinkedList
Contagem de frequências	HashMap<K, Long>	$\Theta(1)$ amort.	Counter map padrão
Top-K elementos	PriorityQueue (min-heap)	$\Theta(n \log k)$	Heap com tamanho fixo k
Deteção de duplicatas	HashSet	$\Theta(1)$ amort.	Sem ordem necessária
Autocompletar por prefixo	Trie	$\Theta(\text{prefixo})$	Estrutura ideal para prefixos

Cache com evicção LRU	LinkedHashMap	$\Theta(1)$ amort.	accessOrder + removeEldest
-----------------------	---------------	--------------------	-------------------------------

4. Implementação em Java

4.1 Pipeline de Benchmarking JMH Completo

A seguir, o pom.xml completo para um projeto de benchmarking com JMH e Spring Boot:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
  </parent>

  <groupId>br.edu.iftm</groupId>
  <artifactId>estrutura-dados-web</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <properties>
    <java.version>21</java.version>
    <jmh.version>1.37</jmh.version>
  </properties>

  <dependencies>
    <!-- Spring Boot -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-cache</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-amqp</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-redis</artifactId>
    </dependency>

    <!-- JMH -->
    <dependency>
      <groupId>org.openjdk.jmh</groupId>
      <artifactId>jmh-core</artifactId>
      <version>${jmh.version}</version>
    </dependency>
  </dependencies>
```

```

        <groupId>org.openjdk.jmh</groupId>
        <artifactId>jmh-generator-annprocess</artifactId>
        <version>${jmh.version}</version>
        <scope>provided</scope>
    </dependency>

    <!-- Test -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
</project>

```

4.2 Benchmark Comparativo: HashMap vs TreeMap vs LinkedHashMap

```

@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@State(Scope.Benchmark)
@Warmup(iterations = 5, time = 1)
@Measurement(iterations = 10, time = 1)
@Fork(2)
public class MapBenchmark {

    private static final int TAMANHO = 100_000;
    private HashMap<String, Integer> hashMap;
    private TreeMap<String, Integer> treeMap;
    private LinkedHashMap<String, Integer> linkedHashMap;
    private List<String> chaves;

    @Setup(Level.Trial)
    public void setup() {
        hashMap = new HashMap<>(TAMANHO);
        treeMap = new TreeMap<>();
        linkedHashMap = new LinkedHashMap<>(TAMANHO);
        chaves = new ArrayList<>(TAMANHO);

        Random rand = new Random(42);
        for (int i = 0; i < TAMANHO; i++) {
            String chave = "produto-" + rand.nextInt(TAMANHO * 10);
            chaves.add(chave);
            hashMap.put(chave, i);
            treeMap.put(chave, i);
            linkedHashMap.put(chave, i);
        }
    }

    @Benchmark
    public Integer hashMapGet(Blackhole bh) {
        String chave =
chaves.get(ThreadLocalRandom.current().nextInt(chaves.size()));
        Integer val = hashMap.get(chave);
        bh.consume(val);
        return val;
    }

    @Benchmark
    public Integer treeMapGet(Blackhole bh) {

```

```

        String chave =
chaves.get(ThreadLocalRandom.current().nextInt(chaves.size()));
        Integer val = treeMap.get(chave);
        bh.consume(val);
        return val;
    }

    @Benchmark
    public Integer linkedHashMapGet(Blackhole bh) {
        String chave =
chaves.get(ThreadLocalRandom.current().nextInt(chaves.size()));
        Integer val = linkedHashMap.get(chave);
        bh.consume(val);
        return val;
    }

    // Benchmark de range query – vantagem do TreeMap
    @Benchmark
    public NavigableMap<String, Integer> treeMapRange(Blackhole bh) {
        NavigableMap<String, Integer> sub =
            treeMap.subMap("produto-1000", true, "produto-2000", true);
        bh.consume(sub);
        return sub;
    }
}

```

5. Análise de Complexidade

Resumo das complexidades das estruturas e padrões abordados nesta unidade:

Operação	Melhor caso	Caso médio	Pior caso
HashMap.get / .put	$\Theta(1)$	$\Theta(1)$ amort.	$O(n)$ colisões
TreeMap.get / .put	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
LinkedHashMap (LRU)	$\Theta(1)$	$\Theta(1)$ amort.	$O(n)$ colisões
Trie.inserir / buscar	$\Theta(s)$	$\Theta(s)$	$\Theta(s)$
Sliding Window Rate Limiter	$O(1)$	$O(1)$	$O(W)$ onde W =janela
Árvore B+ (busca)	$\Theta(\log_m n)$	$\Theta(\log_m n)$	$\Theta(\log_m n)$
Árvore B+ (range scan)	$\Theta(\log_m n + k)$	$\Theta(\log_m n + k)$	$\Theta(\log_m n + k)$

Onde:

- n = número de elementos na estrutura
- m = ordem da árvore B (número de filhos por nó, tipicamente 100–1000 em SGBDs)
- $|s|$ = comprimento da string (chave na Trie)
- k = número de elementos retornados no range scan
- W = tamanho da janela de tempo no rate limiter (número de requisições armazenadas)



Por que o TreeMap é mais lento que o HashMap na prática?

Apesar do HashMap ter $\Theta(1)$ amortizado vs $\Theta(\log n)$ do TreeMap, em hardware real o TreeMap pode sofrer penalidades de cache miss pois percorre nós de Red-Black Tree espalhados na heap.

O HashMap, com localidade de memória superior (array interno), costuma ter melhor desempenho de CPU cache para conjuntos pequenos. O JMH revela essas nuances que a notação Big-O não captura.

6. Comparação com o Java Collections Framework

Esta unidade conecta as implementações próprias com as implementações de referência do JCF:

Estrutura Própria	Equivalente JCF	Quando usar a própria
LRUCache<K, V>	LinkedHashMap<K, V>	Nunca — LinkedHashMap já implementa LRU
Trie	Nenhum equivalente direto	Sempre — JCF não tem Trie
SlidingWindowRateLimiter	Nenhum equivalente	Sempre — lógica de domínio
FilaPrioridade<T>	PriorityQueue<T>	Fins pedagógicos; use JCF em produção
HashTable<K, V>	HashMap<K, V>	Fins pedagógicos; HashMap é mais eficiente
BST<T>	TreeMap<K, V> / TreeSet<T>	Fins pedagógicos; JCF usa Red-Black Tree

7. Testes com JUnit 5

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

class LRUCacheTest {

    private Map<String, Integer> cache;

    @BeforeEach
    void setUp() {
        cache = LRUCache.create(3); // capacidade 3
    }

    @Test
    @DisplayName("Deve armazenar e recuperar um valor")
    void deveArmazenarERecuperar() {
        cache.put("a", 1);
        assertEquals(1, cache.get("a"));
    }

    @Test
    @DisplayName("Deve respeitar a capacidade máxima")
    void deveRespeitarCapacidade() {
        cache.put("a", 1);
        cache.put("b", 2);
        cache.put("c", 3);
        cache.put("d", 4); // "a" deve ser removido (LRU)

        assertEquals(3, cache.size());
        assertNull(cache.get("a"), "a deveria ter sido eviccionado");
        assertEquals(4, cache.get("d"));
    }

    @Test
    @DisplayName("Deve manter elemento recém-acessado no cache")
    void deveManterElementoAcessado() {
        cache.put("a", 1);
        cache.put("b", 2);
        cache.put("c", 3);
        cache.get("a"); // "a" torna-se o mais recente
        cache.put("d", 4); // "b" deve ser removido (agora o LRU)

        assertNotNull(cache.get("a"), "a foi acessado - não deve ser eviccionado");
        assertNull(cache.get("b"), "b deve ter sido eviccionado");
    }
}

class RateLimiterTest {

    private RateLimiter rateLimiter;

    @BeforeEach
    void setUp() {
        rateLimiter = new RateLimiter();
    }
}
```

```

@Test
@DisplayName("Deve permitir requisições dentro do limite")
void devePermitirDentroDoLimite() {
    for (int i = 0; i < 100; i++) {
        assertTrue(rateLimiter.permitir("cliente-1"),
            "Requisição " + i + " deve ser permitida");
    }
}

@Test
@DisplayName("Deve bloquear a 101ª requisição")
void deveBloquearApos100() {
    for (int i = 0; i < 100; i++) {
        rateLimiter.permitir("cliente-1");
    }
    assertFalse(rateLimiter.permitir("cliente-1"),
        "101ª requisição deve ser bloqueada");
}

@Test
@DisplayName("Clientes diferentes têm limites independentes")
void clientesSaoIndependentes() {
    for (int i = 0; i < 100; i++) {
        rateLimiter.permitir("cliente-1");
    }
    assertTrue(rateLimiter.permitir("cliente-2"),
        "cliente-2 não deve ser afetado pelo limite do cliente-1");
}
}

class TrieTest {

    private Trie trie;

    @BeforeEach
    void setUp() {
        trie = new Trie();
        trie.inserir("java");
        trie.inserir("javascript");
        trie.inserir("jpa");
        trie.inserir("json");
        trie.inserir("python");
    }

    @Test
    @DisplayName("Deve retornar sugestões por prefixo")
    void deveRetornarSugestoesPorPrefixo() {
        List<String> resultados = trie.autocompletar("ja", 10);
        assertEquals(3, resultados.size());
        assertTrue(resultados.containsAll(List.of("java", "javascript",
"jpa")));
    }

    @ParameterizedTest
    @ValueSource(strings = {"xyz", "kotlin", "rust"})
    @DisplayName("Deve retornar lista vazia para prefixo inexistente")
    void deveRetornarVazioParaPrefixoInexistente(String prefixo) {
        assertTrue(trie.autocompletar(prefixo, 10).isEmpty());
    }
}

```

```
    }

    @Test
    @DisplayName("Deve respeitar o limite máximo de resultados")
    void deveRespeitarLimiteMaximo() {
        List<String> resultados = trie.autocompletar("j", 2);
        assertEquals(2, resultados.size());
    }
}
```


8. Exercícios Práticos

Exercício 1 — Análise de Benchmark JMH

Execute o benchmark MapBenchmark apresentado na Seção 4.2 e responda:

- Qual estrutura teve menor latência média na operação get? O resultado era esperado pela análise de complexidade?
- Qual foi a variância (erro) das medições? O que isso indica sobre a previsibilidade das estruturas?
- Adicione um benchmark para ArrayList.get(index) vs LinkedList.get(index) com listas de 10.000 elementos. Explique o resultado.

Resolução (Exercício 1 — análise esperada):

Esperamos $\text{HashMap} < \text{LinkedHashMap} \approx \text{TreeMap}$ para o get aleatório, com HashMap sendo 2–5x mais rápido que TreeMap. A variância é tipicamente baixa para operações in-memory sem GC. O `LinkedList.get(i)` deve ser ordens de magnitude mais lento ($O(n)$) que `ArrayList.get(i)` ($O(1)$) para índices ao meio da lista — este resultado demonstra na prática o pitfall clássico de uso de `LinkedList` com acesso indexado.

Exercício 2 — LRU Cache para Catálogo de Produtos

Implemente um serviço Spring Boot que:

- Mantém um cache LRU local com capacidade de 500 produtos.
- Expõe o endpoint GET `/api/produtos/{id}` que retorna o produto em cache ou busca no banco (simulado).
- Expõe o endpoint GET `/api/cache/stats` retornando quantas entradas há no cache e a taxa de hit/miss (use contadores `AtomicLong`).

Resolução (Exercício 2 — estrutura esperada):

```
@RestController
@RequestMapping("/api/produtos")
public class ProdutoController {
    @Autowired private ProdutoService service;

    @GetMapping("/{id}")
    public ResponseEntity<Produto> buscar(@PathVariable Long id) {
        return ResponseEntity.ok(service.buscarComCache(id));
    }
}

@Service
public class ProdutoService {
    private final Map<Long, Produto> cache = LRUCache.create(500);
    private final AtomicLong hits = new AtomicLong();
    private final AtomicLong misses = new AtomicLong();

    public Produto buscarComCache(Long id) {
        Produto p = cache.get(id);
        if (p != null) { hits.incrementAndGet(); return p; }
        misses.incrementAndGet();
        p = buscarNoBanco(id); // simulado
        cache.put(id, p);
        return p;
    }
}
```

```

    }

    public Map<String, Object> stats() {
        long h = hits.get(), m = misses.get(), total = h + m;
        return Map.of("entradas", cache.size(), "hits", h, "misses", m,
            "hitRate", total == 0 ? 0.0 : (double) h / total);
    }
}

```

Exercício 3 — Rate Limiter por IP com Redis

Modifique o RateLimiter da Seção 3.5.1 para usar Redis como backend (em vez de ConcurrentHashMap). Vantagem: funciona com múltiplas instâncias do serviço (escala horizontal). Use o comando INCR do Redis com EXPIRE.

Resolução (Exercício 3):

```

@Component
public class RedisRateLimiter {
    @Autowired
    private RedisTemplate<String, String> redisTemplate;

    private static final int MAX = 100;

    public boolean permitir(String clienteId) {
        String chave = "rate:" + clienteId;
        Long count = redisTemplate.opsForValue().increment(chave);
        if (count != null && count == 1) {
            redisTemplate.expire(chave, Duration.ofMinutes(1));
        }
        return count != null && count <= MAX;
    }
}

```

9. Desafios para Casa

Desafio 1 — Análise de GC em Aplicação Spring Boot (Dificuldade: Média)

Configure uma aplicação Spring Boot com os flags de GC logging (-Xlog:gc*). Execute uma carga de 10.000 requisições usando Apache JMeter ou wrk. Analise o gc.log e responda:

- Qual é a frequência média de GC durante a carga?
- As pausas de GC afetam a latência das requisições?
- Qual seria o impacto de reduzir o heap máximo (-Xmx) à metade?

Desafio 2 — Pipeline de Notificações com RabbitMQ (Dificuldade: Alta)

Implemente o sistema de notificações completo:

- Producer: endpoint POST /api/transacoes que salva a transação e publica um evento.
- Consumer: listener que processa o evento e "envia um e-mail" (log no console).
- Dead Letter Queue: configure uma DLQ para mensagens que falham após 3 tentativas.
- Retry: implemente reproprocessamento com backoff exponencial (1s, 2s, 4s).

Desafio 3 — Sistema de Autocompletar com Trie Persistente (Dificuldade: Alta)

Expanda o AutoCompleter:

- Adicione peso/frequência a cada palavra (mais buscada = prioridade maior).
- Retorne resultados ordenados por frequência (use PriorityQueue<T> no retorno).
- Persista os pesos em Redis e recarregue-os no @PostConstruct.

10. Armadilhas Clássicas (Pitfalls)

⚠ Cache com objetos mutáveis

Armazenar objetos mutáveis no cache sem clonagem pode causar inconsistência: quem modifica o objeto recebido modifica o objeto em cache. Solução: retornar cópias defensivas ou usar objetos imutáveis (records do Java 21).

⚠ ConcurrentModificationException no rate limiter

Iterar sobre um HashMap enquanto outra thread insere causa ConcurrentModificationException. Solução: usar ConcurrentHashMap ou sincronizar acesso com synchronized.

⚠ Cache inconsistente após atualização no banco

Atualizar o banco sem invalidar o cache (cache stale) causa leituras de dados desatualizados. Solução: sempre usar @CacheEvict ou @CachePut nas operações de escrita.

⚠ Benchmark JMH sem @Fork(2)

Sem fork, JMH executa na mesma JVM do processo de construção, sofrendo interferência de outras classes. Sempre usar @Fork(2) ou mais.

⚠ LinkedList com get(i) em loop

for (int i=0; i<lista.size(); i++) { lista.get(i); } é $O(n^2)$ para LinkedList. Use Iterator ou enhanced for. Este é o pitfall mais comum de uso incorreto de estruturas.

⚠ Redis sem TTL

Chaves Redis sem TTL crescem indefinidamente e consomem memória até o Redis aplicar a política de evicção (LRU, LFU). Sempre definir TTL ao inserir no Redis.

⚠ N+1 Query com Hibernate + Cache

Habilitar cache de segundo nível do Hibernate sem configurar corretamente o provider pode resultar em queries duplicadas ou dados inconsistentes. Prefira Spring Cache explícito para controle total.

11. Conexão com o Projeto Integrador

Esta apostila preparou todos os fundamentos para o Projeto Integrador da Semana 17. Você implementará um sistema web completo ("EduMetrics API") que integra:

- Cache Redis com Spring Cache para dados de desempenho dos alunos.
- Rate Limiting por IP para proteger os endpoints.
- Autocompletar de disciplinas com Trie em memória.
- Filas RabbitMQ para geração assíncrona de relatórios.
- Benchmarking JMH comparando as estruturas implementadas ao longo do curso.

O projeto integrador é o culminar da jornada de 80 horas: cada estrutura estudada nas Unidades I–III reaparece em um sistema real, com análise de performance comprovada por métricas.

12. Referências

1. BLOCH, J. Effective Java. 3. ed. Boston: Addison-Wesley, 2018.
2. CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.
3. ORACLE CORPORATION. JMH: Java Microbenchmark Harness. Disponível em: <https://openjdk.org/projects/code-tools/jmh/>. Acesso em: maio 2026.
4. REDIS LTD. Redis Documentation. Disponível em: <https://redis.io/docs/>. Acesso em: maio 2026.
5. PIVOTAL SOFTWARE. Spring Framework Documentation — Cache Abstraction. Disponível em: <https://docs.spring.io/spring-framework/docs/current/reference/html/integration.html#cache>. Acesso em: maio 2026.
6. RABBITMQ TEAM. RabbitMQ Documentation. Disponível em: <https://www.rabbitmq.com/documentation.html>. Acesso em: maio 2026.
7. BAUER, C.; KING, G.; GREGORY, G. Java Persistence with Hibernate. 2. ed. Shelter Island: Manning, 2015.
8. HUNT, A.; THOMAS, D. The Pragmatic Programmer: Your Journey to Mastery. 20th Anniversary ed. Boston: Addison-Wesley, 2019.

13. Glossário

Benchmark	Medição controlada e repetível do desempenho de um fragmento de código.
Cache Miss	Evento em que um dado solicitado não é encontrado no cache, forçando busca na fonte original.
Cache Hit	Evento em que um dado solicitado é encontrado no cache, evitando acesso à fonte original.
Dead Letter Queue (DLQ)	Fila especial para mensagens que falharam no processamento após todas as tentativas de retry.
Evicção	Remoção de uma entrada do cache para liberar espaço. Políticas comuns: LRU, LFU, FIFO.
JIT (Just-In-Time Compilation)	Compilação de bytecode Java para código nativo em tempo de execução, realizada pela JVM para otimizar código frequentemente executado.
LRU (Least Recently Used)	Política de evicção que remove o elemento acessado há mais tempo.
Profiling	Análise dinâmica de uma aplicação em execução para identificar consumo de CPU e memória.
Pub/Sub	Padrão de comunicação onde producers publicam mensagens em canais e consumers assinam esses canais sem acoplamento direto.
Range Scan	Consulta que retorna todos os registros cujo valor de chave está em um intervalo [min, max].
Routing Key	Chave de roteamento do AMQP que determina para qual fila uma mensagem publicada em uma Exchange será enviada.
TTL (Time-To-Live)	Tempo de vida de uma entrada em cache; após o TTL, a entrada expira automaticamente.
Trie	Árvore de prefixos onde cada caminho da raiz a um nó terminal representa uma string armazenada.
Warm-up	Fase inicial de execuções descartadas em benchmarking, para permitir que a JVM realize otimizações JIT antes das medições.